

Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования «Санкт-Петербургский национальный исследовательский университет информационных технологий, механики и оптики».

Факультет программной инженерии и компьютерной техники.

Лабораторная работа №4

465755 вариант

Выполнила:

Джантуре Назерке Жасланқызы; Группа № Р3108

Проверила:

Заболотняя Ольга Михайловна

Санкт-Петербург 2025 г.

Содержание

<u>Задание</u>	3
<u>SQL скрипт</u>	4-6
<u>Заключение</u>	7

Задание

Составить запросы на языке SQL (пункты 1-2).

Для каждого запроса предложить индексы, добавление которых уменьшит время выполнения запроса (указать таблицы/атрибуты, для которых нужно добавить индексы, написать тип индекса; объяснить, почему добавление индекса будет полезным для данного запроса).

Для запросов 1-2 необходимо составить возможные планы выполнения запросов. Планы составляются на основании предположения, что в таблицах отсутствуют индексы. Из составленных планов необходимо выбрать оптимальный и объяснить свой выбор.

Изменяются ли планы при добавлении индекса и как?

Для запросов 1-2 необходимо добавить в отчет вывод команды EXPLAIN ANALYZE [запрос]

Подробные ответы на все вышеперечисленные вопросы должны присутствовать в отчете (планы выполнения запросов должны быть нарисованы, ответы на вопросы - представлены в текстовом виде).

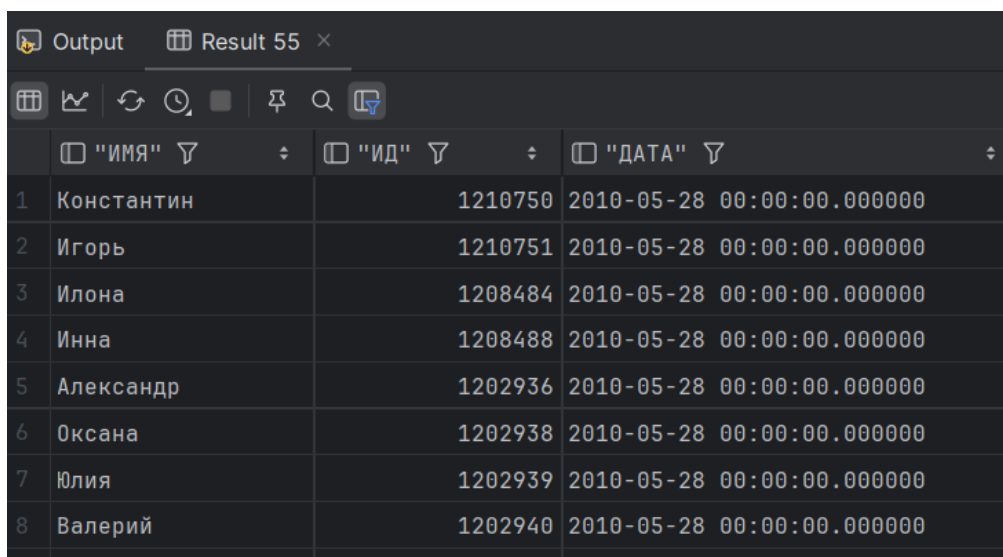
1. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:
Таблицы: Н_ЛЮДИ, Н_ВЕДОМОСТИ.
Вывести атрибуты: Н_ЛЮДИ.ИМЯ, Н_ВЕДОМОСТИ.ИД.
Фильтры (AND):
а) Н_ЛЮДИ.ИД < 142095.
б) Н_ВЕДОМОСТИ.ДАТА = 2010-05-28.
с) Н_ВЕДОМОСТИ.ДАТА > 1998-01-05.
Вид соединения: INNER JOIN.
2. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:
Таблицы: Н_ЛЮДИ, Н_ОБУЧЕНИЯ, Н_УЧЕНИКИ.
Вывести атрибуты: Н_ЛЮДИ.ФАМИЛИЯ, Н_ОБУЧЕНИЯ.ЧЛВК_ИД, Н_УЧЕНИКИ.НАЧАЛО.
Фильтры: (AND)
а) Н_ЛЮДИ.ИД < 121961.
б) Н_ОБУЧЕНИЯ.ЧЛВК_ИД = 117938.
Вид соединения: LEFT JOIN.

SQL скрипт

[\\1\\](#)

```
SELECT "Н_ЛЮДИ"."ИМЯ", "Н_ВЕДОМОСТИ"."ИД",  
"Н_ВЕДОМОСТИ"."ДАТА"  
FROM "Н_ЛЮДИ"  
      INNER JOIN "Н_ВЕДОМОСТИ" ON "Н_ЛЮДИ"."ИД" =  
"Н_ВЕДОМОСТИ"."ЧЛВК_ИД"  
WHERE "Н_ЛЮДИ"."ИД" < 142095  
      AND DATE("Н_ВЕДОМОСТИ"."ДАТА") = '2010-05-28'  
      AND DATE("Н_ВЕДОМОСТИ"."ДАТА") > '1998-01-05';
```

Выполнение:



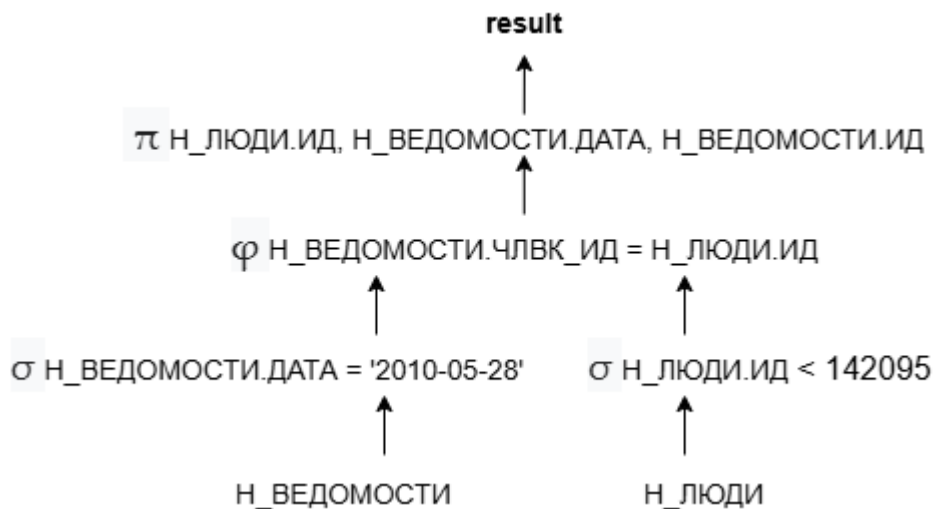
	"ИМЯ" ▾	"ИД" ▾	"ДАТА" ▾
1	Константин	1210750	2010-05-28 00:00:00.000000
2	Игорь	1210751	2010-05-28 00:00:00.000000
3	Илона	1208484	2010-05-28 00:00:00.000000
4	Инна	1208488	2010-05-28 00:00:00.000000
5	Александр	1202936	2010-05-28 00:00:00.000000
6	Оксана	1202938	2010-05-28 00:00:00.000000
7	Юлия	1202939	2010-05-28 00:00:00.000000
8	Валерий	1202940	2010-05-28 00:00:00.000000

Планы выполнения:

№1



№2



Оптимальным является план №2, так как он производит объединение таблиц по уже выбранным ранее атрибутам, а не по таблицам целиком.

Индексы

```
CREATE INDEX "ИНД_ЛЮДИ_ИД" ON "Н_ЛЮДИ" USING BTREE("ИД");  
CREATE INDEX "ИНД_ЛЮДИ_ИМЯ" ON "Н_ЛЮДИ" USING HASH("ИМЯ");  
CREATE INDEX "ИНД_ВЕД_ДАТА" ON "Н_ВЕДОМОСТИ" USING HASH("ДАТА");  
CREATE INDEX "ИНД_ВЕД_ИД" ON "Н_ВЕДОМОСТИ" USING HASH("ИД");
```

Выборка происходит с использованием операторов сравнения, поэтому оптимально использование BTREE. При этом для операции равенства оптимально использование HASH.

Добавление этих индексов должно ускорить выполнение запросов, так как по перечисленным полям происходит выборка с использованием оператора сравнения, а также теперь соединение таблиц будет происходить быстрее.

При добавлении индексов планы выполнения запросов изменятся, так как вместо полного сканирования таблиц будет производиться индексный скан и Hash Join станет быстрее.

EXPLAIN ANALYZE

EXPLAIN ANALYZE

```
SELECT "ИМЯ", "Н_ВЕДОМОСТИ"."ИД", "ДАТА"  
FROM "Н_ЛЮДИ"  
INNER JOIN "Н_ВЕДОМОСТИ"  
ON "Н_ЛЮДИ"."ИД" = "Н_ВЕДОМОСТИ"."ЧЛВК_ИД"  
WHERE "ДАТА" = '2010-05-28'  
AND DATE("ДАТА") > '1998-01-05'  
AND "Н_ЛЮДИ"."ИД" < 142095;
```

Hash Join (cost=207.76..418.86 rows=48 width=25) (actual time=1.640..1.754 rows=57 loops=1)

" Hash Cond: (""Н_ВЕДОМОСТИ"". ""ЧЛВК_ИД"" = ""Н_ЛЮДИ"". ""ИД"")"

" -> Index Scan using ""ВЕД_ДАТА_I"" on ""Н_ВЕДОМОСТИ"" (cost=0.29..211.21 rows=71 width=16) (actual time=0.029..0.113 rows=144 loops=1)"

" Index Cond: ((""ДАТА"" > '1998-01-05 00:00:00'::timestamp without time zone)
AND (""ДАТА"" = '2010-05-28 00:00:00'::timestamp without time zone))"

-> Hash (cost=163.97..163.97 rows=3479 width=17) (actual time=1.574..1.575 rows=3473 loops=1)

Buckets: 4096 Batches: 1 Memory Usage: 206kB

" -> Seq Scan on ""Н_ЛЮДИ"" (cost=0.00..163.97 rows=3479 width=17) (actual time=0.009..0.950 rows=3473 loops=1)"

" Filter: (""ИД"" < 142095)"

Rows Removed by Filter: 1645

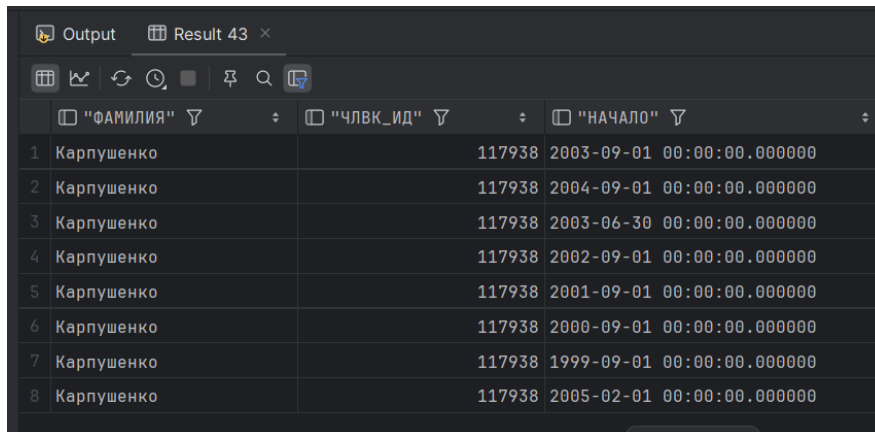
Planning Time: 0.759 ms

Execution Time: 1.802 ms

\\2\\

```
SELECT "Н_ЛЮДИ"."ФАМИЛИЯ", "Н_ОБУЧЕНИЯ"."ЧЛВК_ИД",  
"Н_УЧЕНИКИ"."НАЧАЛО"  
FROM "Н_ЛЮДИ"  
LEFT JOIN "Н_ОБУЧЕНИЯ" ON "Н_ОБУЧЕНИЯ"."ЧЛВК_ИД" =  
"Н_ЛЮДИ"."ИД"  
LEFT JOIN "Н_УЧЕНИКИ" ON "Н_УЧЕНИКИ"."ЧЛВК_ИД" =  
"Н_ОБУЧЕНИЯ"."ЧЛВК_ИД"  
WHERE "Н_ЛЮДИ"."ИД" < 121961  
AND "Н_ОБУЧЕНИЯ"."ЧЛВК_ИД" = '117938';
```

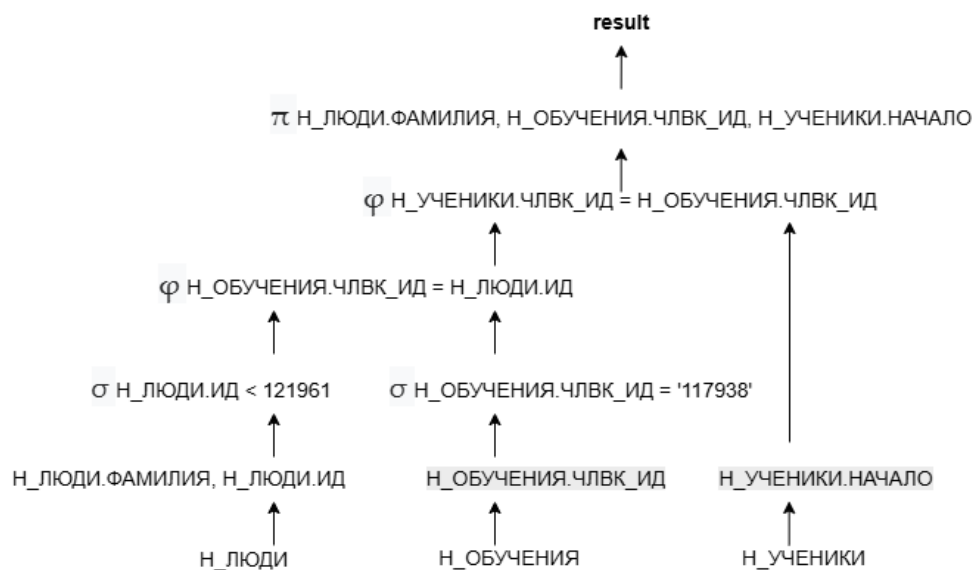
Выполнение:



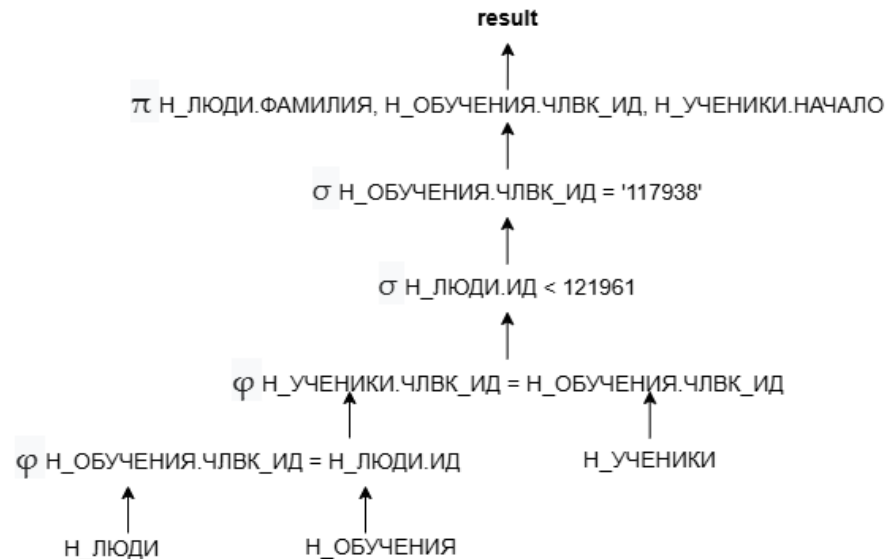
	"ФАМИЛИЯ"	"ЧЛВК_ИД"	"НАЧАЛО"
1	Карпушенко	117938	2003-09-01 00:00:00.000000
2	Карпушенко	117938	2004-09-01 00:00:00.000000
3	Карпушенко	117938	2003-06-30 00:00:00.000000
4	Карпушенко	117938	2002-09-01 00:00:00.000000
5	Карпушенко	117938	2001-09-01 00:00:00.000000
6	Карпушенко	117938	2000-09-01 00:00:00.000000
7	Карпушенко	117938	1999-09-01 00:00:00.000000
8	Карпушенко	117938	2005-02-01 00:00:00.000000

Планы выполнения:

№1



№2



Индексы

```
CREATE INDEX "ИНД_ЛЮДИ_ИД" ON "H_ЛЮДИ" USING BTREE("ИД");
CREATE INDEX "ИНД_ЛЮДИ_ФАМ" ON "H_ЛЮДИ" USING HASH("ФАМИЛИЯ");
CREATE INDEX "ИНД_УЧ_НАЧ" ON "H_УЧЕНИКИ" USING HASH("НАЧАЛО");
CREATE INDEX "ИНД_ОБУЧ_ЧЛВК" ON "H_ОБУЧЕНИЯ" USING HASH("ЧЛВК_ИД");
```

Выборка происходит с использованием операторов сравнения, поэтому оптимально использование BTREE. При этом для операции равенства оптимально использование HASH.

Добавление этих индексов должно ускорить выполнение запросов, так как по перечисленным полям происходит выборка с использованием оператора сравнения, а также теперь соединение таблиц будет происходить быстрее.

При добавлении индексов планы выполнения запросов изменятся, так как вместо полного сканирования таблиц будет производиться индексный скан и Nested Loops Join станет быстрее.

EXPLAIN ANALYZE

EXPLAIN ANALYZE

```
SELECT "ФАМИЛИЯ", "Н_ОБУЧЕНИЯ"."ЧЛВК_ИД", "НАЧАЛО"  
FROM "Н_ЛЮДИ"  
    LEFT JOIN "Н_ОБУЧЕНИЯ" ON "Н_ОБУЧЕНИЯ"."ЧЛВК_ИД" =  
    "Н_ЛЮДИ"."ИД"  
    LEFT JOIN "Н_УЧЕНИКИ" ON "Н_УЧЕНИКИ"."ЧЛВК_ИД" =  
    "Н_ОБУЧЕНИЯ"."ЧЛВК_ИД"  
WHERE "Н_ОБУЧЕНИЯ"."ЧЛВК_ИД" = '117938'  
AND "Н_ЛЮДИ"."ИД" < 121961;
```

Nested Loop Left Join (cost=4.89..35.65 rows=5 width=28) (actual time=0.098..0.146 rows=8 loops=1)

-> Nested Loop (cost=0.56..12.61 rows=1 width=20) (actual time=0.048..0.050 rows=1 loops=1)

" -> Index Scan using ""ЧЛВК_ПК"" on ""Н_ЛЮДИ"" (cost=0.28..8.30 rows=1 width=20) (actual time=0.010..0.011 rows=1 loops=1)"

" Index Cond: ((""ИД"" < 121961) AND (""ИД"" = 117938))"

" -> Index Only Scan using ""ОБУЧ_ЧЛВК_FK_I"" on ""Н_ОБУЧЕНИЯ"" (cost=0.28..4.30 rows=1 width=4) (actual time=0.033..0.034 rows=1 loops=1)"

" Index Cond: (""ЧЛВК_ИД"" = 117938)"

Heap Fetches: 0

" -> Bitmap Heap Scan on ""Н_УЧЕНИКИ"" (cost=4.33..22.99 rows=5 width=12) (actual time=0.046..0.090 rows=8 loops=1)"

" Recheck Cond: (""ЧЛВК_ИД"" = 117938)"

Heap Blocks: exact=7

" -> Bitmap Index Scan on ""УЧЕН_ОБУЧ_FK_I"" (cost=0.00..4.32 rows=5 width=0) (actual time=0.039..0.039 rows=8 loops=1)"

" Index Cond: (""ЧЛВК_ИД"" = 117938)"

Planning Time: 48.431 ms

Execution Time: 0.189 ms

Заключение

В ходе работы над данной лабораторной я по лучше стала разбираться в командах SQL, по лучше узнала про индексы и такие функции как EXPLAIN ANALYZE, Nested Loop и так далее.